TASK 11 – Recommend and Defend the Final Design

Recommendation

Retain the modular, function-based design, but use Decimal values constructed from input strings for all currency and quality calculations. Keep the rate card in one module or configuration object, validate every entry, and persist each delivery record for audit. Round the rate and payment at the policy-defined settlement points using an explicitly selected rounding mode. Integer paise is a valid alternative for final money totals when all values are already expressed to whole paise.

Rate revision

A rate revision should be made in one central rate-card location, tested against known examples, versioned, and applied with an effective date. Historical payments should retain the rate card that applied when the delivery was recorded; changing the current rate must not silently recalculate old payments.

Limitations

The original specification gives a fat range but does not specify an SNF validation range, seasonal effective-date rules, missing analyser readings, duplicate transactions, or a precise settlement rounding policy. Those business rules must be supplied by the co-operative. A simple in-memory script is not sufficient for concurrent clerks, access control, backups or audit history; a database-backed application is required for production use.

District-union scale

The arithmetic is O(1) per delivery and processing n deliveries is O(n). At 150,000 entries, this amount of computation is practical in Python. Storing every delivery requires O(n) space, which is also manageable with appropriate database storage. For a district union, use database transactions, unique transaction identifiers, backups, validation at the data boundary, logging and reproducible rate-card versions. Performance should be measured using representative data rather than assumed.

Alternative rate file

Reading t configuration values from JSON at start-up takes O(t) time and O(t) memory. This allows rate changes without editing the source module, but the file must be validated, protected from unauthorised edits, versioned and backed up. For a small static rate card, a Python module is simple; for seasonal and administrator-managed changes, a validated JSON configuration or database table is more flexible. The choice depends on the operational need for rate updates and governance.